SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3

6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3
8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3

15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3

16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3
19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3

24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I RenugaDevi B(192311111) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: RenugaDevi B

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Testing as an Engineering Activity

Random testing is similar to checking a building by walking around and looking for cracks without an inspection plan. A startup should replace this with an engineering workflow. First, requirements should be converted into test conditions. Then suitable techniques such as equivalence partitioning, boundary value analysis, decision tables and state-transition testing should be selected. For an online registration system, instead of randomly entering values, testers can divide age into valid and invalid groups, test minimum and maximum limits, and check combinations such as invalid OTP + expired session. A traceability link can be maintained as: Requirement → Test condition → Test case → Execution result → Defect Automation can handle repetitive regression tests, while exploratory testing can investigate unusual behavior. Thus, structured testing improves defect detection because testing effort is directed toward risk, requirements and meaningful input combinations, rather than depending on chance.

2. Defined Testing Process in a Banking System

A banking system cannot depend on individual testers following their own methods because even a small transaction error can affect money. A standard process can begin with requirement review, followed by test planning, test design, environment preparation, execution, defect reporting, retesting and regression testing. Consider fund transfer. Requirements first define amount limits, beneficiary validation and transaction status. Testers then create normal, negative and interruption scenarios.

For example, if the network disconnects after the customer's account is debited, the system must not create an uncertain transaction state. A defined process provides repeatability, traceability and measurable quality. Every release is evaluated using the same entry and exit criteria. Therefore, standardization reduces forgotten scenarios and makes banking software more dependable.

3. Structured Testing Through SDLC

Testing should move alongside development rather than appearing as the final activity. SDLC Phase Testing Activity Requirements Review requirements for ambiguity Design Prepare scenarios and test conditions Coding Unit-level verification Integration Check communication between modules System Validate complete workflows Acceptance Verify business expectations Maintenance Regression testing For an e-commerce application, the product module may work independently while the cart incorrectly receives product prices. Integration testing exposes this problem. Process discipline also defines who tests, what is tested, when it is tested and what evidence is required.

Consequently, defects are discovered closer to their source, integration problems are reduced and the final product becomes more predictable.

4. Error, Defect and Failure

Consider a login requirement stating that an account must be blocked after five consecutive incorrect passwords. A developer misunderstands the requirement and writes a limit of ten attempts. This human misunderstanding is an error. The incorrect five-attempt rule implemented in the program becomes a defect. When a user enters the sixth wrong password and the system still allows another attempt, the user observes a failure. Therefore: Error → introduces Defect → Defect executes → Failure becomes visible The distinction matters because the solution differs. Testing exposes the failure, while investigation identifies the defect and its originating error. Clear defect reports should describe both the expected and actual behavior instead of simply stating “login failed.”

5. Early Defect Detection Principle

The most useful principle here is early testing, often called shift-left testing. Suppose an application requires password length between 8 and 20 characters. If the tester reviews this requirement before coding, an ambiguity such as “Does exactly 8 count?” can be resolved immediately. If the same issue is discovered after deployment, it may require code modification, retesting, documentation changes and customer support.

The tester can participate in requirement reviews, design reviews and testability discussions. Thus: Requirement defect → detected early → corrected cheaply instead of: Requirement defect → implemented → discovered late → expensive correction Early involvement improves effectiveness because it prevents defects rather than merely finding them after implementation.

6. Tester Contribution During Healthcare Requirements

Healthcare requirements must be precise because incorrect interpretation can affect important patient information. Suppose the requirement says: “Patient records should be automatically archived after a long period.” A tester should immediately question what “long period” means. Is it one year, five years or ten years? Can doctors access archived records? Can an archived record be modified? The tester converts vague statements into testable conditions. Activities include: – Reviewing requirement ambiguity – Identifying missing scenarios – Checking acceptance criteria – Asking boundary-related questions – Identifying data and security risks

Early tester participation prevents unclear requirements from becoming expensive implementation defects.

7. Origin of Payroll Defects

A payroll defect can originate much earlier than the line of code where it appears. For example, suppose the requirement says: “Employees receive overtime payment according to company policy.” This is incomplete because the actual overtime rate is not specified. The ambiguity may travel through: Unclear requirement → incorrect design assumption → incorrect code → wrong salary → production defect A tester can prevent this by requesting precise rules such as overtime hours, rate, holidays and maximum limits. Requirements should contain measurable acceptance criteria. For example: Overtime Pay = Overtime Hours × Approved Overtime Rate Requirement reviews and example-based validation can expose contradictions before development. Therefore, many software defects are actually lifecycle defects, not merely programming mistakes.

8. Boundary Input Crash

If a system crashes only when boundary values are entered, the defect can be classified as a boundary-related functional defect, potentially caused by an input validation or programming error.

Suppose a system accepts quantities from 1 to 100. Testing only 20 and 50 does not adequately examine the boundary. A stronger set is: 0, 1, 2, 99, 100, 101 Here, 1 and 100 are valid boundaries, while 0 and 101 test values immediately outside them. Boundary Value Analysis is effective because programming mistakes frequently occur around limits. Had these values been included in the original test design, the crash could have been detected before release.

9. Defect Repository as a Test Design Tool

A defect repository should function like a learning system, not merely an archive. Suppose previous releases contain: – 10 payment defects – 7 discount defects – 2 product-search defects The tester should not give all three modules equal regression attention. Historical information can identify: high-defect module → high-risk condition → additional test cases If payment failures frequently occur when transactions are interrupted, future regression suites should contain network interruption and duplicate-submission tests.

Repository fields such as root cause, module, severity, release and defect type help identify patterns. Thus, old defects become inputs for new test design, reducing repeated failures.

10. ATM Withdrawal as a Functional Defect

Suppose an ATM receives a request to withdraw ■2,000 but deducts ■3,000 from the account. The ATM has accepted a valid operation but produced an incorrect business result. This is a functional defect. The tester should compare: Input: ■2,000 Expected debit: ■2,000 Actual debit: ■3,000 Prevention requires more than testing one successful withdrawal. Test cases should include different denominations, account balances, withdrawal limits and repeated withdrawals. Boundary and decision-table testing can verify combinations such as: Sufficient balance + valid amount + valid PIN → withdrawal permitted Structured transaction testing would likely expose the incorrect calculation before deployment.

11. Defect Clustering

Defect clustering means defects tend to accumulate in particular parts of a system rather than being evenly distributed. Suppose a large application contains 15 modules, but 65% of historical defects come from three modules: payment, reporting and authentication. Testing resources should therefore be weighted toward those modules.

A simple priority approach is: Historical defects + business criticality + complexity = testing priority Payment may receive intensive regression, integration and negative testing, while a low-risk information page receives basic functional checks. This does not mean ignoring other modules. It means recognizing that equal testing effort does not necessarily produce equal risk coverage.

12. Slow Web Application Under Heavy Load

If a web application becomes extremely slow when thousands of users access it simultaneously, the issue is primarily a performance defect. It may result from inefficient database queries, insufficient server resources, poor caching or unoptimized application logic. Different testing can expose different behavior: – Load testing: expected number of users – Stress testing: beyond normal capacity – Spike testing: sudden user increase – Endurance testing: prolonged load For example, an application may respond in two seconds for 100 users but take 30 seconds for 5,000 users. Classification helps because developers can investigate performance bottlenecks instead of treating the issue as an ordinary functional failure.

13. Historical Defects in Regression Testing

Historical defects provide a ready-made source of regression scenarios.

Suppose earlier versions had failures involving: – Invalid coupon codes – Duplicate payments – Expired sessions – Incorrect tax calculation These defects should not disappear from the testing process after being fixed. Their test cases should enter the regression suite. The process becomes: Old defect → permanent test case → future release → automatic execution Frequently repeated defects can receive automation priority. This improves efficiency because testers do not have to rediscover the same scenarios manually in every release. Historical data therefore becomes a practical source of risk-based regression coverage.

14. Tester Role in Integration Defects

A tester should act as a quality bridge between requirements, development and integration teams. Suppose the developer sends a date as: DD-MM-YYYY while another service expects: YYYY-MM-DD Each module may pass its own unit tests, but integration fails. An early tester can identify the expected interface contract during design discussions.

The tester contributes by: – Reviewing interfaces – Preparing integration scenarios – Clarifying assumptions – Checking API contracts – Participating in defect analysis Early collaboration prevents “works in my module” situations and improves end-to-end quality.

15. Preventing Recurring Defects

If the same defect appears every release, fixing it repeatedly is not enough. The organization should identify why the defect keeps being introduced. Suppose incorrect discount calculations appear in three releases. The team should investigate: Why was it introduced? → Why was it not detected? → Why did regression not catch it? Possible preventive actions include: – Clarifying requirements – Adding calculation-based tests – Performing code reviews – Automating regression cases – Adding boundary tests – Conducting root-cause analysis The objective changes from defect correction to defect prevention. This creates a stronger quality cycle where lessons from one release actively reduce defects in the next.

16. Standardizing Mobile Application Testing

A mobile application can behave differently across devices, operating systems and network conditions. Therefore, testers should use a standardized test strategy. A device matrix can cover: OS version × screen size × device type × network condition Common scenarios should include installation, login, permissions, backgrounding, notifications, low network and battery conditions. Each test can follow the same lifecycle: Prepare → Execute → Record → Report → Retest → Regression Automation can handle repetitive compatibility tests, while exploratory testing can investigate unusual device behavior. Standardization ensures that one tester does not test only Wi-Fi while another tests only mobile data.

17. Testing vs Debugging

Imagine an online payment displays “Payment Failed”, but the customer's money has already been deducted. The tester's responsibility is to reproduce the problem, collect evidence and report the failure. The developer's debugging activity begins afterward. The developer examines logs, database transactions and payment-service responses to determine why the incorrect status was generated. Therefore: Testing = finding and demonstrating the problem. Debugging = locating and correcting the cause.

If developers treat debugging as testing, they may verify only the corrected code and miss unrelated defects. A proper cycle is: Test → Defect Report → Debug → Fix → Retest → Regression Both activities support quality, but their objectives are different.

18. Cross-Browser Testing

An online shopping site should not assume that successful Chrome testing means the product is browser-compatible. Testing should select browsers based on customer usage and business importance. For example: Area Test UI Layout and alignment JavaScript Buttons and dynamic actions Cart Quantity and price updates Payment Secure transaction flow Responsive design Different screen sizes Automation tools such as Selenium or Playwright can execute common workflows across browsers. If the checkout button works in Chrome but becomes unresponsive in Firefox, the tester should record the browser, version, operating system and reproduction steps. A browser matrix combined with automation gives broader coverage with less repetitive effort.

19. Validation Gaps in Logistics Software

Improper validation allows incorrect data to travel deeper into the system. Suppose a logistics application accepts a delivery date earlier than the order date. If validation is missing, the system may calculate an impossible delivery schedule. Validation should occur at appropriate levels: Input → Business rule → Database → Integration → Output Testers should deliberately provide invalid, incomplete, extreme and contradictory values. For example: – Empty tracking ID – Invalid tracking ID – Already delivered shipment – Past delivery date – Impossible destination combination Proper validation testing prevents incorrect data from entering later processing stages and improves system reliability.

20. Systematic Test Case Design

Random testing asks: “What can I try?” Engineering-based testing asks: “What evidence do I need to show that this requirement works?”

For a password feature, test design can be derived from requirements. If the allowed length is 8–16 characters, tests should include: 7, 8, 9, 15, 16, 17 characters Additional classes can cover empty input, spaces, special characters and incorrect passwords. Techniques such as equivalence partitioning, boundary analysis and decision tables make the selection deliberate. The result is not necessarily a larger test suite. It is a more intelligent test suite, where each test has a reason for existing.

21. Data Inconsistency Between Healthcare Modules

Suppose the patient's address is changed in the registration module but the billing module continues using the previous address. The tester should trace the data through: UI → API → Database → Service → Receiving Module Possible causes include incorrect API mapping, stale cached data, failed synchronization or database transaction problems. Testing should include: – Create data in one module – Modify it – Read it from another module – Verify consistency – Test simultaneous updates – Test failed transactions and rollback Data comparison tools and API testing can help identify where the values diverge.

Thus, integration testing is not only about whether modules communicate—it must also verify that they exchange correct and consistent information.

22. Tester Role Across SDLC

The tester's role should evolve throughout the SDLC. Requirements: identify ambiguity and missing acceptance criteria. Design: evaluate testability and interfaces. Development: prepare test data and automation. Integration: verify module communication. System testing: validate complete workflows. Acceptance: confirm business expectations. Maintenance: perform regression testing. For example, if testers participate during design of a payment module, they can ask what happens when payment succeeds but the confirmation response is lost. Such questions may prevent an expensive production defect. Therefore, early tester involvement changes testing from a final inspection activity into continuous quality engineering.

23. Failure During Peak Usage

A system that works with normal traffic but fails during peak usage has a testing coverage gap. Suppose an online examination system supports 2,000 students normally but 10,000 students submit answers simultaneously.

Testing should cover: – Normal load – Expected peak load – Sudden traffic spike – Long-duration usage – Database connection limits – Concurrent submissions A coverage matrix should map these scenarios to requirements and risks. For example, if “10,000 simultaneous submissions” is a critical business scenario and no test case exists for it, there is a clear coverage gap. Better coverage therefore means covering important conditions, not simply increasing the number of test cases.

24. Defect Repository and Documentation

Poor documentation makes defects behave like lost files—they may exist but cannot be effectively used. A useful repository should capture: ID → Module → Description → Steps → Expected → Actual → Severity → Priority → Environment → Evidence → Root Cause → Status For example, “Checkout failure” is too vague. A stronger report states exactly what input caused the problem, what should have happened, what actually happened and under which environment. The repository can later be searched for defects by module or category. This supports both current tracking and future prevention, because historical defect information becomes available to new testers and future projects.

25. Redesigning Banking Transaction Testing

For banking transactions, testing should be organized around risk rather than screens. The highest-risk scenarios are those that can cause financial loss or duplicate transactions. The test strategy should include: 1. Normal transaction 2. Insufficient balance 3. Invalid beneficiary 4. Maximum transaction amount 5. Network interruption 6. Duplicate submission 7. Server timeout 8. Rollback after failure 9. Concurrent transactions 10. Incorrect authentication For example, if the network fails after debit but before confirmation, the system must preserve a consistent transaction state. Automation can repeatedly execute regression scenarios, while exploratory testing can investigate unusual transaction sequences. This approach tests the behavior of money movement, not merely whether buttons work.

26. Missing Edge Cases

Normal-value testing might use: Age = 25, 30, 40 But an age field with a valid range of 18–60 requires attention to its edges. Useful values are:

17, 18, 19, 59, 60, 61 Boundary Value Analysis targets precisely these transition points. Edge cases should be identified from: Minimum values Maximum values – Empty values – Zero – Negative values – Maximum length – Resource limits Including them in both functional and regression suites improves defect detection because many implementation mistakes occur at boundaries rather than in ordinary values.

27. Process-Based Testing for Consistency

Different modules may have different functionality, but the quality process should remain consistent. A common organizational framework can define: Test planning → Test design → Review → Execution → Defect logging → Retest → Regression → Closure For example, the payment team and profile team may use completely different test cases, but both should follow the same rules for severity, evidence and defect reporting. Checklists can ensure essential activities are not skipped. Periodic peer reviews can also compare test practices across teams. Standardization therefore creates uniformity in process without forcing identical test cases, allowing specialized testing while maintaining consistent quality control.

28. Unexpected Input Handling

Unexpected input should be treated as a planned testing category, not as an afterthought. Consider a username field expected to accept letters and numbers. Testers should try: – Empty input – Spaces – Special symbols – Very long strings – Unicode characters – SQL-like input – Repeated characters The objective is not merely to check whether an error message appears. The tester should verify that the application remains stable and does not expose sensitive information. Input validation should occur at both client and server levels where appropriate. A robust system should respond to invalid input predictably instead of crashing or producing corrupted data.

29. Preventing Defect Leakage into Production

Frequent production defects usually indicate a weakness somewhere in the quality pipeline. The first step is to classify escaped defects by: Module → Severity → Root Cause → Detection Stage Suppose most production defects come from integration problems. Increasing unit tests alone will not solve the problem. The organization should strengthen integration testing, API validation and end-to-end scenarios. Other improvements include: – Risk-based test planning – Requirement reviews – Regression automation Production-like test environments Exploratory testing – Root-cause analysis – Release exit criteria The objective is to create a defect containment strategy, where problems are detected at the earliest practical stage instead of reaching customers.

30. Redesigning Testing for Early Defect Detection

The testing strategy should be redesigned as a sequence of quality checkpoints. Checkpoint 1 – Requirements: remove ambiguity. Checkpoint 2 – Design: verify architecture and interfaces. Checkpoint 3 – Code: perform unit and code-level checks. Checkpoint 4 – Integration: validate communication. Checkpoint 5 – System: test complete workflows. Checkpoint 6 – Regression: ensure fixes did not create new problems. Checkpoint 7 – Release: verify exit criteria. For example, if an e-commerce discount rule is incorrect, discovering it during requirement review costs far less than discovering it after deployment. Therefore, early detection reduces rework, debugging effort, retesting time and release risk. The strongest strategy is not simply “test more.” It is to place the right quality activity at the right stage, while using risk and historical defects to decide where deeper testing is necessary.